HOMEWORK 1: BACKPROPAGATION AND NEURAL NETWORKS

[Zhang Zeyu]
Student ID: **[2026213422]**
Deep Learning (84100343-0)
Tsinghua University, Autumn 2026

ABSTRACT

This coursework report includes the author's handwritten backpropagation solutions, the completed NumPy expert module, and a controlled comparison of dense and sparse mixture-of-experts language models.

1 PART ONE: BACKPROPAGATION

The next two pages reproduce the handwritten solutions which cover Tasks 1.3.1–1.3.4. The pseudo code is listed as the end.

Deep Learning HW1. Zhang Zeyu. 20262134422 the answers are highlighted by △

1.3.1

$\because \vec{s} = (2,1,-1,3)^T \quad \vec{q} = (1, \frac{1}{2}, \frac{1}{2})^T \Rightarrow a_1 = (2,1,-1)\begin{pmatrix}1\\ \frac{1}{2}\\ \frac{1}{2}\end{pmatrix} = 2 \quad , \quad a_2 = (1,-1,3)\begin{pmatrix}1\\ \frac{1}{2}\\ \frac{1}{2}\end{pmatrix} = 2$

$\therefore \underset{\triangle}{h} = ReLU(\vec{a}) = \begin{pmatrix}2\\ 2\end{pmatrix}$ (1)*

let $(\vec{v}_1, \vec{u}_1, \vec{f}_1), (\vec{v}_2, \vec{u}_2, \vec{f}_2)$ as the intermediate quantity of the experts, than:

$$\begin{cases} \vec{v}_1 = W_{1,1}\vec{h} + \vec{b}_{1,1} = \begin{pmatrix}2\\ -\frac{3}{2}\end{pmatrix}, \ \vec{u}_1 = \begin{pmatrix}2\\ 0\end{pmatrix}, \ \vec{f}_1 = W_{2,1}\vec{u}_1 + \vec{b}_{2,1} = \begin{pmatrix}\frac{2}{5}\\ \frac{1}{2}\end{pmatrix} \\ \vec{v}_2 = W_{1,2}\vec{h} + \vec{b}_{1,2} = \begin{pmatrix}2\\ -1\end{pmatrix}, \ \vec{u}_2 = \begin{pmatrix}2\\ 0\end{pmatrix}, \ \vec{f}_2 = W_{2,2}\vec{u}_2 + \vec{b}_{2,2} = \begin{pmatrix}\frac{1}{5}\\ \frac{2}{5}\end{pmatrix} \end{cases} \quad (2)^*$$

the logits and gate weights of the router are calculated as:

$$\vec{q} = Softmax(G\vec{h}) = Softmax\left(\begin{pmatrix}\frac{1}{4} & \frac{1}{4}\\ 0 & 0\end{pmatrix}\begin{pmatrix}2\\ 2\end{pmatrix}\right) = Softmax\left(\begin{pmatrix}0\\ 0\end{pmatrix}\right) = \begin{pmatrix}\frac{1}{2}\\ \frac{1}{2}\end{pmatrix}$$

$$\Rightarrow \begin{cases} \vec{r} = \vec{h} + \sum_{e=1}^{2} q_e \vec{f}_e = \vec{h} + q_1\vec{f}_1 + q_2\vec{f}_2 = \begin{pmatrix}\frac{69}{20}\\ \frac{69}{20}\end{pmatrix}, \ C\vec{r} + \vec{b}_c \Rightarrow \begin{pmatrix}\frac{1}{5} & 0\\ 0 & \frac{1}{5}\\ \frac{1}{5} & \frac{1}{5}\end{pmatrix}\begin{pmatrix}\frac{69}{20}\\ \frac{69}{20}\end{pmatrix} + \begin{pmatrix}\frac{69}{100}\\ \frac{69}{100}\\ \frac{98}{100}\end{pmatrix} = \begin{pmatrix}0\\ 0\\ 0\end{pmatrix} \Rightarrow \hat{y} = \begin{pmatrix}\frac{1}{3}\\ \frac{1}{3}\\ \frac{1}{3}\end{pmatrix} \quad (3)^* \\ \mathcal{L} = -\frac{1}{m}\sum_{i=1}^{m}\sum_{k=1}^{k} y_k^{(i)} \log \hat{y}_{(k)}^{(i)} \xrightarrow{m=1} -\sum_{k=1}^{3} y_k \log \hat{y}_k \xrightarrow[\vec{y}=(0,1,0)^T]{} y_2 \log \hat{y}_2 = \log 3 \approx 1.09861. \end{cases}$$

1.3.2

let $z \in \mathbb{R}^k$ is the logits of sample $k \in \mathbb{R}^k$

let $p_i = \frac{e^{z_i}}{\sum_{j=1}^{k} e^{z_j}}$ is the probability after Softmax, than:

$$\therefore \begin{cases} \frac{\partial e^{z_i}}{\partial z_j} = \begin{cases} e^{z_i}, & i=j \\ 0, & i \neq j \end{cases} = \vec{1}[i=j]e^{z_i} \\ \frac{\partial Z}{\partial z_j} = \frac{\partial}{\partial z_j}\sum_{c=1}^{k} e^{z_c} = e^{z_j} \end{cases} \Rightarrow \frac{\partial p_i}{\partial z_j} = \frac{Z\frac{\partial e^{z_i}}{\partial z_j} - e^{z_i}\frac{\partial Z}{\partial z_j}}{Z^2} = \vec{1}[i=j]\frac{e^{z_i}}{Z} - \frac{e^{z_i}}{Z}\cdot\frac{e^{z_j}}{Z} = p_i(\vec{1}[i=j] - \vec{p}_j) = \begin{cases} p_i(1-p_i), & i=j \\ -p_i p_j, & i \neq j \end{cases}$$

let Jacobian matrix $\vec{J}_{ij} = \frac{\partial p_i}{\partial z_j} \in \mathbb{R}^{k\times k} (i: 1\to k, j: 1\to k) \Rightarrow \underset{\triangle}{\vec{J}} = diag(\vec{p}) - \vec{p}\vec{p}^T$

$$\therefore \begin{cases} \vec{J}^T = \vec{J} \ (1) \\ \vec{p}^T\vec{1} = \sum_i \vec{p}_i = 1 \Rightarrow \vec{J}\vec{1} = diag(\vec{p})\vec{1} - \vec{p}(\vec{p}^T\vec{1}) = \vec{p} - \vec{p} = \vec{0} \ (2) \end{cases}$$

let cross-entropy $\ell = -\sum_{a=1}^{k} y_a \log p_a$, $\frac{\partial \ell}{\partial z_b} = \sum_{a=1}^{k}\frac{\partial \ell}{\partial p_a}\frac{\partial p_a}{\partial z_b} = \sum_a (-\frac{y_a}{p_a})p_a(\vec{1}[a=b] - p_b) = -\sum_a y_a\vec{1}[a=b] + \sum_a y_a p_b$

$$\xrightarrow[\sum_a y_a p_b = p_b\sum_a y_a]{\sum_a y_a\vec{1}[a=b] = y_b} \frac{\partial \ell}{\partial z_b} = -y_b + p_b\sum_a y_a \xrightarrow{\sum_a y_a = 1} \frac{\partial \ell}{\partial z_b} = p_b - y_b, \ \underset{\triangle}{\nabla_z \ell} = p - y \in \mathbb{R}^k \ (3)$$

when the mini batch enters. let the batch has m samples:

the average loss $\mathcal{L} = \frac{1}{m}\sum_{i=1}^{m}\ell^{(i)} \Rightarrow \nabla_{z^{(i)}}\mathcal{L} = \frac{1}{m}\nabla_{z^{(i)}}\ell^{(i)} \overset{(3)}{=\!=} (p^{(i)} - y^{(i)})\underset{\triangle}{\frac{1}{m}}$, so there's no need for another average.

1.3.3

for general m, D, H, E, K, let $\vec{t}^{(i)} = G\vec{h}^{(i)}$, $\vec{z}^{(i)} = C\vec{r}^{(i)} + b_c$. for sample i,

let $\beta^{(i)} := C^T\vec{\alpha}^{(i)} \in \mathbb{R}^D$, ① then $\nabla_C \mathcal{L} = \sum_i \vec{\alpha}^{(i)} (\vec{r}^{(i)})^T \in \mathbb{R}^{K\times D}$, $\nabla_{b_c}\mathcal{L} = \sum_i \vec{\alpha}^{(i)} \in \mathbb{R}^K$

for expert e, $r_d = h_d + \sum_e g_e f_{e,d} \Rightarrow \frac{\partial r_d}{\partial f_{e,j}} = g_e \mathbb{I}[d=j] \Rightarrow \frac{\partial \mathcal{L}}{\partial f_{e,j}} = \sum_{d=1}^D \frac{\partial \mathcal{L}}{\partial r_d}\frac{\partial r_d}{\partial f_{e,j}} = \sum_d \beta_d g_e \mathbb{I}[d=j] = g_e\beta_j$

$\therefore \eta_e = \nabla_{f_e}\mathcal{L} = g_e\vec{\beta} \in \mathbb{R}^D$

$\vec{f} = W_2\vec{u} + b_2 \Rightarrow f_d = \sum_{j=1}^H (W_2)_{dj} u_j + (b_2)_d$ (the 2nd layer) $\xrightarrow{\frac{\partial f_d}{\partial (W_2)_{dj}} = u_j}$ $\frac{\partial \mathcal{L}}{\partial (W_2)_{dj}} = \frac{\partial \mathcal{L}}{\partial f_d}\cdot\frac{\partial f_d}{\partial (W_2)_{dj}} = \eta_d u_j = (\vec{\eta}\vec{u}^T)_{dj}$

$\Rightarrow \frac{\partial \mathcal{L}}{\partial u_j} = \sum_{d=1}^D \frac{\partial \mathcal{L}}{\partial f_d}\frac{\partial f_d}{\partial u_j} = \sum_d \eta_d (W_2)_{dj} \Rightarrow \nabla_u \mathcal{L} = W_2^T\vec{\eta}$

$\because ReLU'(0) = 0 \quad \therefore \frac{\partial u_j}{\partial v_j} = \mathbb{I}[v_j > 0] \Rightarrow \frac{\partial \mathcal{L}}{\partial v_j} = \frac{\partial \mathcal{L}}{\partial u_j}\mathbb{I}[v_j>0] \Rightarrow \delta := \nabla_v \mathcal{L} = (W_2^T\vec{\eta}) \odot \vec{\mathbb{I}}[v>0] \in \mathbb{R}^H$

$\vec{v} = W_1\vec{h} + b_1 \Rightarrow v_j = \sum_{d=1}^D (W_1)_{jd} h_d + (b_1)_j \Rightarrow \frac{\partial \mathcal{L}}{\partial (W_1)_{jd}} = \delta_j h_d$, $\frac{\partial \mathcal{L}}{\partial (b_1)_j} = \delta_j$ (the 1st layer)

$\therefore \nabla_{W_{2,e}}\mathcal{L} = \sum_i \eta_e^{(i)} (u_e^{(i)})^T \in \mathbb{R}^{D\times H}$, $\nabla_{b_{2,e}}\mathcal{L} = \sum_i (\eta_e^{(i)}) \in \mathbb{R}^D$

$\nabla_{W_{1,e}}\mathcal{L} = \sum_i \delta_e^{(i)} (h^{(i)})^T \in \mathbb{R}^{H\times D}$, $\nabla_{b_{1,e}}\mathcal{L} = \sum_i (\delta_e^{(i)}) \in \mathbb{R}^H$

$\Rightarrow \nabla_{h^{(i)}}\mathcal{L}|_{EP} = \sum_e W_{1,e}^T \delta_e^{(i)}$ ②

for router path, $t = Gh$, $\frac{\partial \mathcal{L}}{\partial h_d}\Big|_{RP} = \sum_{e=1}^E \frac{\partial \mathcal{L}}{\partial t_e}\frac{\partial t_e}{\partial h_d} = \sum_e \rho_e G_{ed}$, $\rho_e := g_e^{(i)}\left(\gamma_e^{(i)} - \sum_j q_j^{(i)}\gamma_j^{(i)}\right)$. $\gamma_e^{(i)} := (f_e^{(i)})^T\beta^{(i)} \in \mathbb{R}$ ③

$\because r(h) = h + \sum_e g_e(h) f_e(h)$, $\therefore dr = dh + \sum_e g_e df_e + \sum_e f_e dg_e \Rightarrow \nabla_{h^{(i)}}\mathcal{L} = \beta^{(i)} + \sum_e W_{1,e}^T\delta_e^{(i)} + G^T\rho^{(i)}$

(residual, expert path, route path)

$\because \vec{\alpha} = \begin{pmatrix} 1/3 \\ -2/3 \\ 1/3 \end{pmatrix}$, $\vec{\beta} = \begin{pmatrix} 2/\sqrt{5} \\ -1/\sqrt{5} \end{pmatrix}$ $\therefore \vec{\gamma} = \begin{pmatrix} 1/\sqrt{10} \\ 1/2\sqrt{3} \end{pmatrix}$, $\vec{\rho} = \begin{pmatrix} 1/2 \\ 1/2 \end{pmatrix} \odot \left[\vec{\gamma} - \frac{3}{100}(1)\right] = \begin{pmatrix} -1/200 \\ 1/200 \end{pmatrix}$ $\therefore \frac{\partial \mathcal{L}}{\partial G_{1,1}} = \rho_1 h_1 = -\frac{1}{100}$

1.3.4

let $\delta_j^{(i)} = \frac{\partial \mathcal{L}}{\partial a_j} = (\nabla_{h^{(i)}}\mathcal{L})_j \mathbb{I}[a_j^{(i)} > 0]$ $\because a_j^{(i)} = \sum_{t=1}^3 q_t s_{j+t-1}^{(i)}$

the gradient of shared kernel $\frac{\partial \mathcal{L}}{\partial q_t} = \sum_{i=1}^m \sum_{j=1}^3 \delta_j^{(i)} s_{j+t-1}^{(i)}$, $t = 1, 2, 3$.

for single enter location l, $\frac{\partial \mathcal{L}}{\partial s_l^{(i)}} = \sum_{\substack{1\le j\le 2 \\ 1\le l-j+1\le 3}} \delta_j^{(i)} q_{l-j+1}$, $\because \delta = \nabla_h \mathcal{L} = (349, -173)^T/2400$

$\nabla_q \mathcal{L} = \frac{1}{2400}\left[349\begin{pmatrix} 2 \\ 1 \\ -1 \end{pmatrix} - 173\begin{pmatrix} 1 \\ -1 \\ 3 \end{pmatrix}\right] = \frac{1}{2400}\begin{pmatrix} 525 \\ 521 \\ -868 \end{pmatrix} = \begin{pmatrix} 7/32 \\ 87/400 \\ -217/600 \end{pmatrix}$, $\frac{\partial \mathcal{L}}{\partial s_2} = \frac{349}{2400}\cdot\frac{1}{2} - \frac{173}{2400}\cdot 1 = \frac{1}{1600}$

give learning rate $\eta = 0.1$, $q_{new} = \begin{pmatrix} 1 \\ 1/2 \\ 1/2 \end{pmatrix} - \frac{1}{10}\begin{pmatrix} 7/32 \\ 87/400 \\ -217/600 \end{pmatrix} = \begin{pmatrix} 313/320 \\ 1913/4000 \\ 3217/6000 \end{pmatrix} \approx (0.978125, 0.478250, 0.536167)^T$.

TASK 1.3.4: MINI-BATCH SGD PSEUDOCODE

The following Python-style pseudocode supplements the convolutional calculations on the preceding handwritten page. It reproduces the corrected code in `results/pseudocode.md`; the forward and backward routines remain abstract. The backward routine returns a fresh gradient dictionary and the full encoded-input gradient, including the residual, expert, and router paths, for each batch.

```
params = {
    "kernel": kernel,
    "expert_w1": expert_w1,  # Includes all experts; likewise for other
    expert parameters.
    "expert_b1": expert_b1,
    "expert_w2": expert_w2,
    "expert_b2": expert_b2,
    "router": router,
    "head": head,
    "head_bias": head_bias,
}
for batch in training_batches:
    prob, cache = forward_all(batch.inputs, params)
    loss = mean_cross_entropy(prob, batch.targets)
    logits_grad = (prob - batch.one_hot_targets) / batch.size
    grads, encoded_grad = backward_head_experts_router_res(logits_grad,
    cache, params)
    conv_grad = encoded_grad * (cache.conv_pre > 0)  # (m, 2)
    # (m, 2, 1) * (m, 2, 3): sum over samples and windows, keeping the
    kernel axis.
    # The upstream gradient already includes 1/m; do not average again.
    grads["kernel"] = (conv_grad[..., None] *
    cache.input_windows).sum(axis=(0, 1))
    for name in params:
        params[name] -= learning_rate * grads[name]
```

Listing 1: Mini-batch SGD for the shared kernel, experts, router, and prediction head.

2 PART TWO: IMPLEMENTING THE EXPERTS

2.1 TASK 2.3.1: FORWARD AND BACKWARD IMPLEMENTATION

Forward pass and array convention. In `NumPy/model.py`, `forward` stores samples as rows of $X \in \mathbb{R}^{B\times D}$, while parameter arrays follow the handout's column-vector convention: $W_{1,e} \in \mathbb{R}^{H\times D}$, $W_{2,e} \in \mathbb{R}^{D\times H}$, $G \in \mathbb{R}^{E\times D}$, and $C \in \mathbb{R}^{K\times D}$ (**?**). With $\mathbf{1}_B$ denoting a column of ones, the batched expert and router computations are

$$V_e = XW_{1,e}^\top + \mathbf{1}_B b_{1,e}^\top, \quad U_e = \max(V_e, 0), \qquad F_e = U_e W_{2,e}^\top + \mathbf{1}_B b_{2,e}^\top, \tag{1}$$

$$S = XG^\top, \qquad A_{ie} = \begin{cases} \dfrac{\exp(S_{ie})}{\sum_{j\in\mathcal{S}_i}\exp(S_{ij})}, & e \in \mathcal{S}_i, \\ 0, & e \notin \mathcal{S}_i, \end{cases} \tag{2}$$

where $\mathcal{S}_i$ is the Top-k set, or the set specified by the supplied Boolean mask. The implementation uses `einsum` to compute all E expert outputs; its masking selects contributions, rather than skipping expert evaluation. Unselected logits are replaced by $-\infty$ before Softmax. Writing $A_{:e}$ as a column vector, residual addition and prediction are

$$R = X + \sum_e A_{:e} \odot F_e, \qquad Z = RC^\top + \mathbf{1}_B b_c^\top, \qquad \mathcal{L} = -\frac{1}{B}\sum_i \log P_{i,y_i}. \tag{3}$$

Here $\odot$ broadcasts the gate column over features and P is the row-wise Softmax of Z. For stability, the code evaluates $\log P_{ic} = \widetilde{Z}_{ic} - \log\sum_j \exp(\widetilde{Z}_{ij})$, with $\widetilde{Z}_{ic} = Z_{ic} - \max_j Z_{ij}$, then caches $P = \exp(\log P)$. The cache keys `v`, `u`, `experts`, `gates`, `residual`, and `probs` correspond to V, U, F, A, R, P.

Backward pass. Let Y be the one-hot encoding of the integer labels, and let an overbar denote the derivative of $\mathcal{L}$ with respect to an array. The NumPy-only `backward` starts with

$$\overline{Z} = (P - Y)/B, \qquad \nabla_C\mathcal{L} = \overline{Z}^\top R, \qquad \nabla_{b_c}\mathcal{L} = \overline{Z}^\top \mathbf{1}_B, \qquad \overline{R} = \overline{Z}C. \tag{4}$$

For each expert, the two `einsum` parameter contractions implement

$$\overline{F}_e = A_{:e} \odot \overline{R}, \qquad \overline{V}_e = (\overline{F}_e W_{2,e}) \odot \mathbf{1}[V_e > 0], \tag{5}$$

$$\nabla_{W_{2,e}}\mathcal{L} = \overline{F}_e^\top U_e, \qquad \nabla_{b_{2,e}}\mathcal{L} = \overline{F}_e^\top \mathbf{1}_B, \tag{6}$$

$$\nabla_{W_{1,e}}\mathcal{L} = \overline{V}_e^\top X, \qquad \nabla_{b_{1,e}}\mathcal{L} = \overline{V}_e^\top \mathbf{1}_B. \tag{7}$$

The router and shared-input gradients are

$$\overline{A}_{ie} = \sum_d \overline{R}_{id}(F_e)_{id}, \qquad \overline{S}_{ie} = A_{ie}\left(\overline{A}_{ie} - \sum_j A_{ij}\overline{A}_{ij}\right), \tag{8}$$

$$\nabla_G\mathcal{L} = \overline{S}^\top X, \qquad \overline{X} = \underbrace{\overline{R}}_{\text{residual}} + \underbrace{\sum_e \overline{V}_e W_{1,e}}_{\text{experts}} + \underbrace{\overline{S}G}_{\text{router}}. \tag{9}$$

These match `d_gates`, `d_router_logits`, and the three `d_input` contributions. The selection mask is held fixed when differentiating, $\mathrm{ReLU}'(0) = 0$, and the factor $1/B$ is applied only once, in `d_logits`. The two-expert numerical case gives loss 1.0986122886681098; the supplied three-expert Top-2 case gives 1.184281870559786.

2.2 TASK 2.3.2: CENTRAL-DIFFERENCE VERIFICATION

The official checker uses float64, perturbation $\epsilon = 10^{-6}$, and a fixed selection mask away from routing ties and ReLU kinks. Table 1 reports the maximum normalized error for every parameter family and the input, using the normalization in the handout. All errors are below the required 10^{-5} threshold.

Table 1: Maximum normalized central-difference errors.

Quantity	Two-expert example	Three-expert Top-2
W_1	1.404e-11	1.370e-10
b_1	1.075e-10	1.234e-10
W_2	4.084e-11	1.173e-10
b_2	9.348e-11	1.000e-10
G	6.077e-11	8.208e-11
C	4.270e-11	1.137e-10
b_c	6.443e-11	1.600e-10
x	4.142e-11	1.083e-10

The supplied faulty variant explicitly subtracts the direct residual contribution from the otherwise complete input gradient. It therefore omits the identity path from the input to the residual sum. Its normalized input error rises to 0.133333 in the two-expert case and 0.279717 in the sparse case. This distinguishes an incorrect shared-input gradient from the parameter gradients, which can still be correct when an input path is omitted.

2.3 Task 2.3.3: Fixed-batch training

Training uses the supplied 128 examples and initialization, fixed context features, 400 full-batch SGD updates, and learning rate 0.2. No hyperparameter search was performed. Training cross-entropy decreases from 5.593877 to 1.870696 (Figure 1). Table 2 shows the three examples selected by the provided runner. Two are incorrect despite the loss reduction; this is a training demonstration, not evidence of generalization.

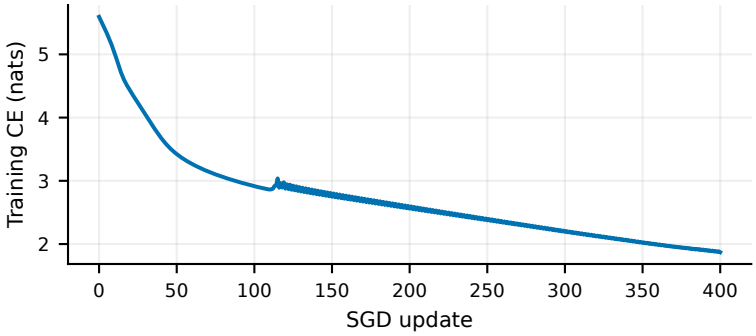


Figure 1: Mean training cross-entropy for the provided NumPy batch. The curve includes the initial loss and the loss after each of 400 SGD updates.

Table 2: Three predictions after 400 NumPy SGD updates. Parentheses contain byte values.

Four-byte context	Target	Prediction
`tle'`	s (115)	space (32)
`est=`	a (97)	e (101)
`u␣as`	space (32)	space (32)

3 PART THREE: A SMALL MOE LANGUAGE MODEL

3.1 TASK 3.4.1: IMPLEMENTATION, TRAINING, AND EVALUATION

Experts, routing, and sparse dispatch. In `MoE/model.py`, `Expert.forward` applies $f_e(h) = W_{2,e}\,\mathrm{ReLU}(W_{1,e}h + b_{1,e}) + b_{2,e}$, a $64 \to 128 \to 64$ MLP. For an FFN input of shape (B, T, D), `FFN.forward` flattens the first two axes to $N = BT$ rows. For token n, `FFN.route` computes

$$s_n = Gh_n, \qquad p_{ne} = \frac{\exp(s_{ne})}{\sum_{j=1}^{E}\exp(s_{nj})}, \tag{10}$$

$$(I_{n1}, \ldots, I_{nk}) = \mathrm{TopKIndices}(s_n), \qquad a_{nr} = \frac{\exp(s_{n,I_{nr}})}{\sum_{q=1}^{k}\exp(s_{n,I_{nq}})}, \tag{11}$$

with $E = 16$ and $k = 2$. The returned `all_probs`, `gates`, and `ids` are respectively $p \in \mathbb{R}^{N\times E}$, $a \in \mathbb{R}^{N\times k}$, and $I \in \{1, \ldots, E\}^{N\times k}$ (mathematical indices are one-based; code IDs are zero-based). Only a weights the expert outputs:

$$o_n = \sum_{r=1}^{k} a_{nr} f_{I_{nr}}(h_n). \tag{12}$$

For each expert e, `dispatch` finds pairs (n, r) with $I_{nr} = e$, gathers those input rows via `index_select`, evaluates f_e, and accumulates $a_{nr}f_e(h_n)$ using `index_add_`. Thus exactly Nk token–expert pairs are evaluated, rather than NE followed by masking.

Residual placement and training objective. The FFN returns o without adding its input. In `MoE/train.py`, `Block.forward` applies pre-normalization and both residuals:

$$\widetilde{X} = X + \mathrm{Attn}_{\mathrm{causal}}(\mathrm{LN}_1(X)), \qquad X^{+} = \widetilde{X} + \mathrm{FFN}(\mathrm{LN}_2(\widetilde{X})). \tag{13}$$

Here the attention term includes the learned output projection. The final LayerNorm and bias-free prediction head produce 256 byte logits per token. For layer ℓ, define its selection fraction and mean full-router probability as

$$q_e^{(\ell)} = \frac{1}{Nk}\sum_{n,r}\mathbf{1}[I_{nr}^{(\ell)} = e], \qquad \overline{p}_e^{(\ell)} = \frac{1}{N}\sum_{n} p_{ne}^{(\ell)}, \qquad \mathcal{A}_\ell = E\sum_{e}\mathrm{stopgrad}(q_e^{(\ell)})\overline{p}_e^{(\ell)}. \tag{14}$$

This is the code's `fraction.detach()` times `probs.mean(0)`, summed and multiplied by E. `Transformer.forward` averages this auxiliary term over its $L = 2$ blocks, so the training loop minimizes

$$\mathcal{L}_{\mathrm{train}} = \underbrace{-\frac{1}{BT}\sum_{b,t}\log P_\theta(y_{bt} \mid x_{b,\le t})}_{\mathcal{L}_{\mathrm{CE}}} + 0.01\,\frac{1}{L}\sum_{\ell=1}^{L}\mathcal{A}_\ell. \tag{15}$$

Targets are the next bytes; dense FFNs return zero auxiliary loss. Gradients are computed by PyTorch autograd. Logged `train_ce` and evaluation CE exclude the auxiliary term.

The official CPU diagnostic uses a separate three-expert fixture and executes exactly 14 selected token–expert pairs for seven tokens with $k = 2$, compared with 21 pairs for dense execution. Its maximum output error is zero and its maximum gradient error is 1.11×10^{-16} against the dense reference. Paired routing records are preserved at each position. Both normal routing and position-wise route perturbation pass prefix invariance, with maximum prefix-logit error zero. These checks establish the tested numerical and causal properties, rather than model quality.

Protocol. The supplied two-block causal context model uses dimension 64, four attention heads, sequence length 64, and a 256-byte vocabulary. The computation-matched dense FFN has hidden width 264; the parameter-matched FFN has width 2056. All runs use seed 416, 3000 updates, and batches of 32 sequences, with equal sampling from Shakespeare and Python. AdamW uses the supplied base learning rate 0.002, weight decay 0.01, cosine schedule with a 0.15 floor fraction, and

gradient clipping at 1. The MoE load-balancing coefficient is 0.01. Common non-FFN initialization and the sampling order are shared across configurations. Runs use PyTorch 2.8.0 on an arm64 CPU with four threads.

Validation is logged every 500 updates using 20 batches per domain; the checkpoint with the lowest validation CE is selected. In all three runs that checkpoint is at update 3000. Figure 2 reports the recorded training and validation curves. Training points are individual current-batch losses, not epoch means; their increase at update 3000 does not imply an increase in validation loss. Held-out test evaluation uses nonoverlapping full-length packs within document boundaries: 111,488 text targets and 1,179,520 code targets. In `evaluate_full`, each domain CE first averages over that domain's targets:

$$\begin{aligned} \mathrm{CE}_d &= -\frac{1}{M_d}\sum_{n=1}^{M_d} \log P_\theta(y_n \mid \mathrm{context}_n), \\ \mathrm{CE} &= \tfrac{1}{2}(\mathrm{CE}_{\mathrm{text}} + \mathrm{CE}_{\mathrm{code}}), \qquad \mathrm{PPL} = \exp(\mathrm{CE}). \end{aligned} \tag{16}$$

The two domains receive equal weight despite different target counts. Test data are not used for checkpoint selection.

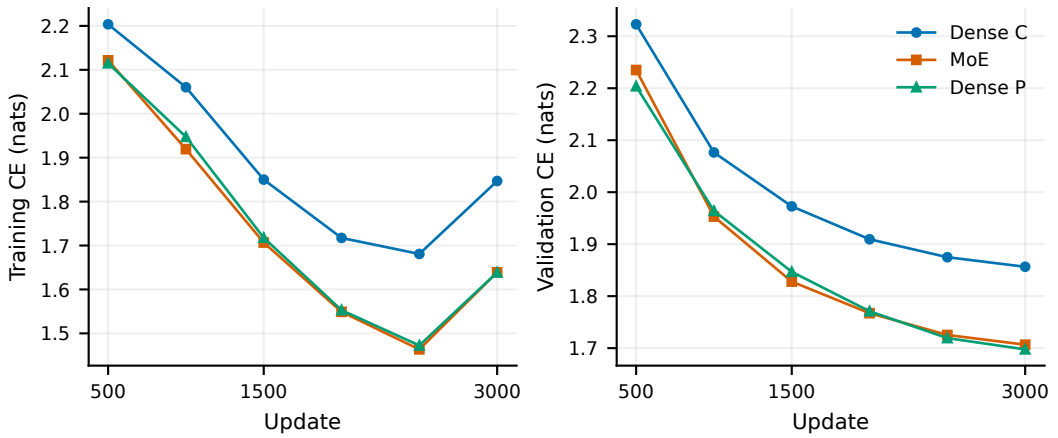


Figure 2: Training and validation cross-entropy for the three configurations under the same 3000-update protocol. Training values are sampled batches at logging steps; validation values average the fixed evaluation procedure. C and P denote computation-matched and parameter-matched dense models.

3.2 Task 3.4.2: Capacity, computation, routing, and usage

Table 3 reports domain metrics, total parameters, matrix MACs per token, and full-batch latency. MAC counts include attention projections and attention products, router, expert matrices, and prediction head across both blocks; they exclude non-matrix operations. With $D = 64$, $T = 64$, $E = 16$, and $k = 2$, the per-token count is

$$2\left[4D^2 + 2TD + F\right] + 256D, \qquad F = \begin{cases} 2DH, & \text{dense}, \\ ED + 2kDH, & \text{MoE}. \end{cases} \tag{17}$$

Latency includes dispatch and uses $B = 32$, $T = 64$, ten warmups, and the median of 30 no-gradient forward passes on the same CPU.

Capacity and quality. At the same 133,120 matrix MACs per token, MoE stores 4.34 times as many parameters as the computation-matched dense model and reduces test CE by 0.1230 nats in this run. Against the parameter-matched dense model, whose parameter count differs by only 0.316%, MoE has similar test CE (difference -0.0032 nats), using about 22.49% of its matrix MACs. This small quality difference from one training seed is not evidence of a reliable advantage.

Table 3: Test results and costs. C/P denote computation/parameter matching; latency is in milliseconds per full batch.

Model	Params	MACs/token	Text CE	Code CE	Mean CE	PPL	ms
Dense C	139,024	133,120	1.9732	1.7985	1.8858	6.5919	2.458
MoE	603,264	133,120	1.8969	1.6289	1.7629	5.8292	5.107
Dense P	601,360	591,872	1.9068	1.6255	1.7661	5.8482	8.083

Measured latency. MoE takes 5.107 ms per batch, compared with 2.458 ms for computation-matched dense and 8.083 ms for parameter-matched dense. Thus equal MACs do not imply equal latency: sparse execution entails gather/scatter, dispatch, and many smaller matrix operations. The measurement is consistent with those overheads but does not isolate their individual costs. The comparison fixes updates and sampling order, rather than wall-clock budget or convergence.

Routing control. The supplied diagnostic exchanges paired expert IDs and gate weights across sequences at the same position in the last layer. It changes neither time positions nor expert–token counts. The trained checkpoint passes prefix invariance for both routing modes, with error zero. The three routing seeds 20261004, 20261005, and 20261006 produce diagnostic CEs of 2.958319, 2.960190, and 2.957595, respectively. Their mean is 2.958701, compared with normal CE 1.762883, an increase of 1.195818. These results indicate sensitivity to the original paired route assignments at this checkpoint. Because predictions depend on other sequences in the batch and the intervention changes both expert IDs and weights, this diagnostic is neither standalone language-model performance nor a causal estimate of the training benefit of routing.

Expert usage and limitations. Figure 3 shows domain-dependent routing frequencies. For example, in the second layer, expert 6 receives about 0.87% of text assignments but 13.59% of code assignments. These frequencies establish different usage patterns, not semantic specialization or an interpretation of what that expert computes. The experiment uses a single model seed, a small fixed corpus, and CPU timing; variation across training seeds and hardware is not measured. The perturbation preserves routing counts but does not preserve each token's assigned experts or gates.

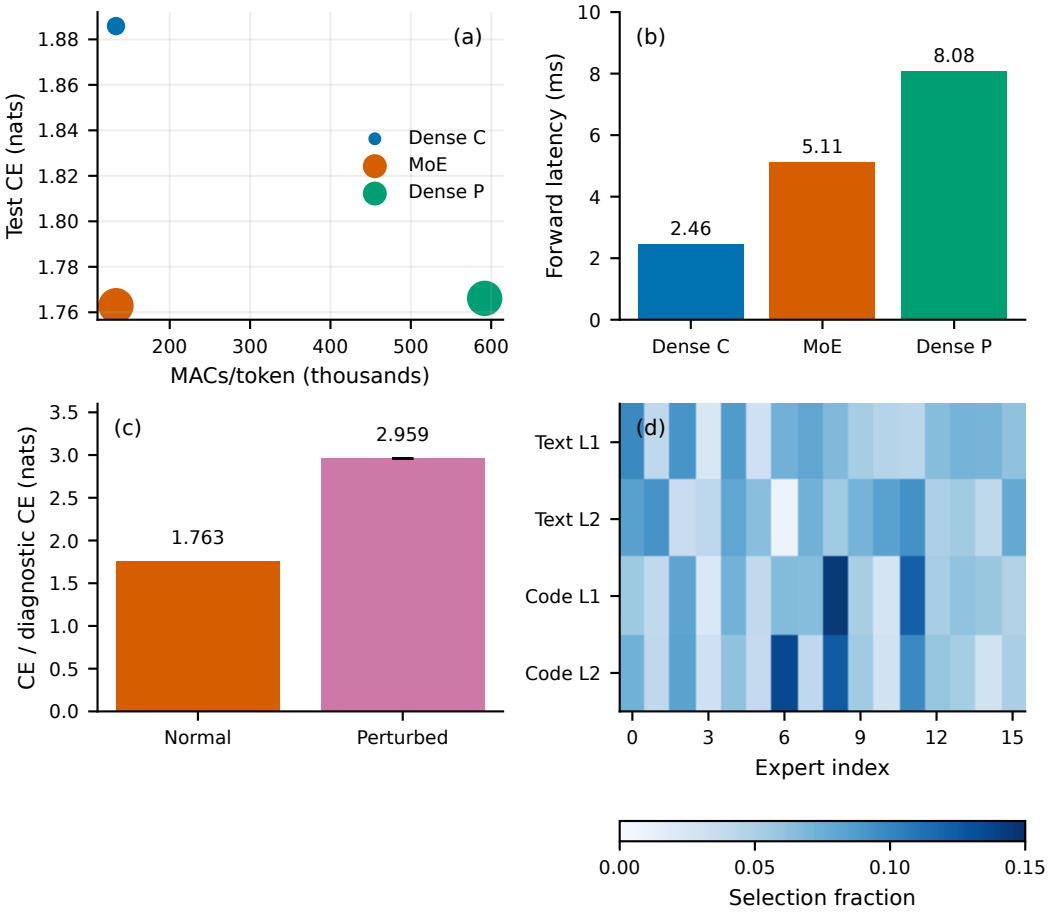


Figure 3: Task 2 comparison. (a) Test quality versus matrix cost; marker area encodes total parameters. (b) Full-batch forward latency including dispatch. (c) Normal CE and the batch-dependent route-perturbation diagnostic; error bars span the three fixed routing seeds, not independent training runs. (d) Expert selection fractions, normalized within each domain and layer. Expert indices are zero-based.

3.3 Task 3: Do learned gate weights matter with fixed experts?

Question and hypothesis. Task 2 changed expert identities and weights together. We ask whether the learned weights still matter when the selected experts are fixed. The hypothesis is that flattening the weights increases CE and reversing their assignment degrades it further, indicating sensitivity to which selected expert receives more weight.

Design and controls. We use the same validation-selected seed-416 checkpoint without retraining. Only the last layer's two gate weights change:

$$\widetilde{g}_e(\lambda) = \tfrac{1}{2} + \lambda(g_e - \tfrac{1}{2}), \qquad e \in \{1, 2\}. \tag{18}$$

The four conditions, specified before evaluation, are original ($\lambda = 1$), uniform (0), midpoint (0.5), and swapped (-1). For two weights summing to one, swapping gives (g_2, g_1) and preserves their multiset, entropy, and norm. It changes their assignment to expert identities. Uniformization additionally changes concentration. All preceding computations, model parameters, ordered expert IDs, and selected token–expert counts remain fixed. Each condition has 603,264 parameters and 133,120 matrix MACs/token. Thus this is a weighting intervention rather than a computation-budget comparison.

Evaluation uses the same disjoint test packs: 111,488 text and 1,179,520 code targets, with batch 32, length 64, and equal domain weighting. The original CE exactly reproduces Task 1. Hashes match for every last-layer input and ordered route across conditions; expert counts and checkpoint tensors also match. Float64 checks verify the weighted-output oracle, 14 executed pairs for seven tokens, prefix invariance, and independence from other batch rows. Unlike Task 2's cross-sequence perturbation, these interventions remain token-local.

Table 4: Last-layer weight intervention on the fixed seed-416 checkpoint. CE is in nats; Δ is relative to original mean CE.

Weights	λ	Text CE	Code CE	Mean CE	Δ CE	PPL
Original	1	1.8969	1.6289	1.7629	+0.0000	5.8292
Uniform	0	2.0792	1.8020	1.9406	+0.1777	6.9631
Midpoint	0.5	1.9298	1.6640	1.7969	+0.0340	6.0309
Swapped	-1	2.8874	2.4569	2.6721	+0.9092	14.4706

Results and interpretation. Table 4 shows that midpoint, uniform, and swapped weights increase mean CE by 0.0340, 0.1777, and 0.9092 nats, respectively; both domains show the same ordering. These measurements support the stated hypothesis at this checkpoint: preserving expert selection alone does not preserve prediction quality. Swapping is particularly damaging despite preserving weight concentration, supporting the importance of matching weights to expert identities rather than concentration alone. It does not establish that the learned weights are optimal over all possible assignments.

Figure 4 provides context: original mean gate gaps are 0.8189 for text and 0.7680 for code, with medians 0.9207 and 0.8919. Only 2.20% and 3.52% of tokens have a gap below 0.1. Thus uniformization and swapping usually make substantial changes, rather than perturbing nearly equal weights. The observed gap distribution is consistent with the sensitivity, but is not itself evidence that larger-gap tokens incur larger loss increases.

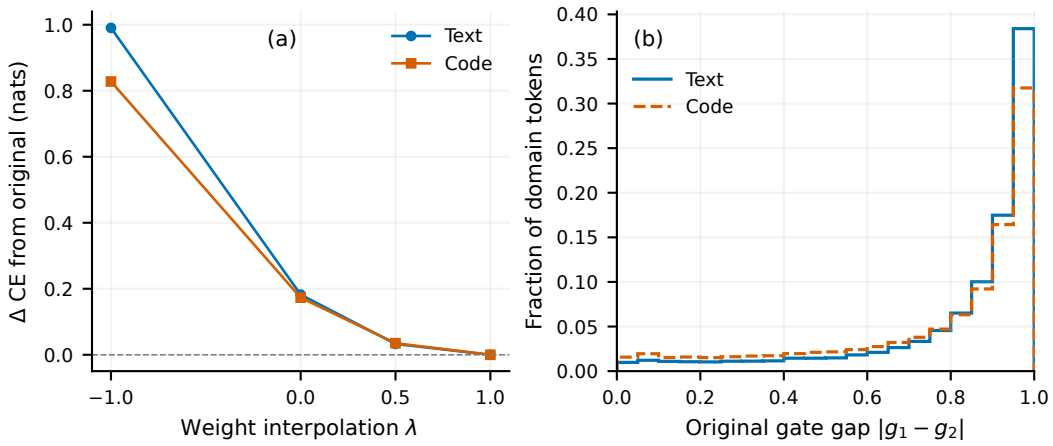


Figure 4: Fixed-expert weight intervention. (a) Domain CE changes at the four specified settings; lines connect evaluated points only. (b) Original last-layer gate-gap distributions, normalized within each domain, with bin width 0.05. These are single-checkpoint measurements, not averages over training seeds.

Limitations and reproducibility. This experiment concerns one checkpoint and the final layer, with no retraining or independent training-seed replication. It does not establish the causal benefit of learning gates during training, nor semantic expert specialization. The four test evaluations describe pre-specified interventions rather than select a new deployed model. Run `MoE/gate_ablation.py` on CPU; the separate `results/gate_ablation.json` records metrics, checks, and source hashes. The AI Use Statement below discloses assistance with this investigation.

Reproducibility and Completion Status

The results follow the provided course handout (**?**). Run the following commands from the release directory, using an environment with the supplied requirements:

```
python NumPy/run.py --check --train
python MoE/diagnostics.py --check
python MoE/train.py --mode all --device cpu
python MoE/diagnostics.py --device cpu
python MoE/gate_ablation.py --device cpu
```

The bundled data snapshots contain the NumPy results (`data/numpy.json`), language-model results (`data/language.json`), and source hashes (`data/run_provenance.json`). Run `prepare_results.py` to rebuild the tables and vector figures. The original logs and selected MoE checkpoint remain in the release's `results/` directory. NumPy results were rerun on the current implementation when preparing this report and reproduce the previously checked values.

AI Use Statement

GPT-6 Astra & Claude Opus 5.5 assisted with reviewing the NumPy and PyTorch implementations, diagnosing errors, and improving variable names, comments, and input checks in the NumPy module. AI also ran the supplied checks and experiments, generated result figures and tables, prepared this LaTeX draft, and helped design, implement, evaluate, and write the author-selected Task 3.4.3 gate-weight investigation. The reported measurements were verified against saved logs, official diagnostics, and intervention-specific controls; AI additionally explained Softmax and expert/router gradients and revised the SGD pseudocode, while the handwritten PDF was supplied by the author and embedded unchanged. All the origin materials and git edit logs could be found on https://github.com/Alchuang22-dev/DLhomework.